

MINERÍA DE DATOS

Maximiliano Ojeda

muojeda@uc.cl



IIC-2433



T-SNE y UMAP

Stochastic Neighbor Embedding (SNE)

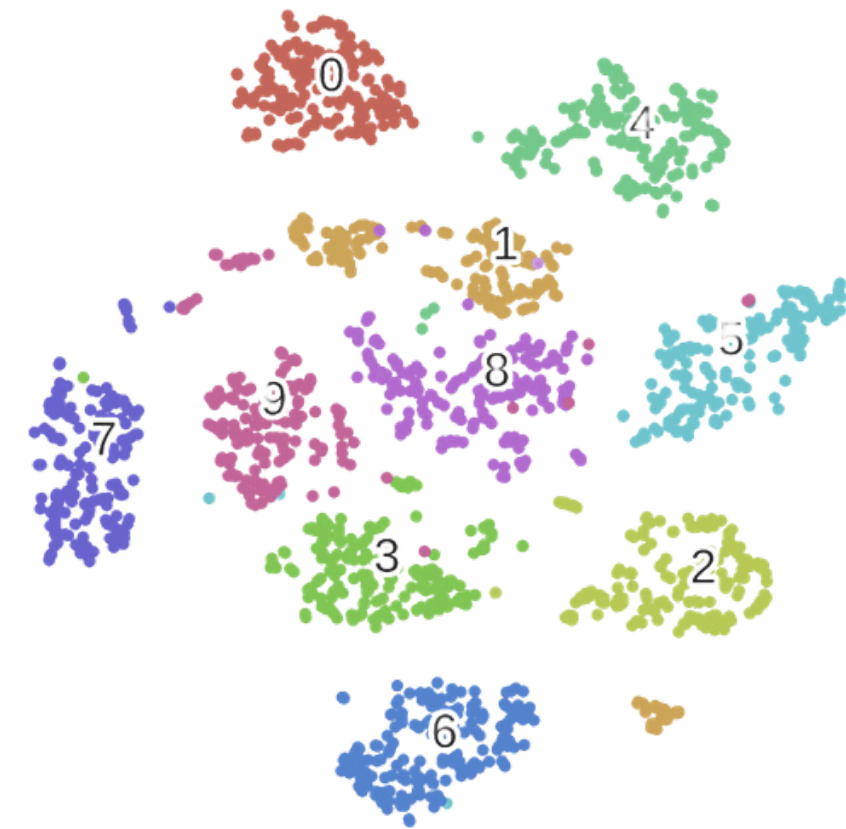
Objetivo

Proyectar datos en 2 o 3 dimensiones para visualización

Problema

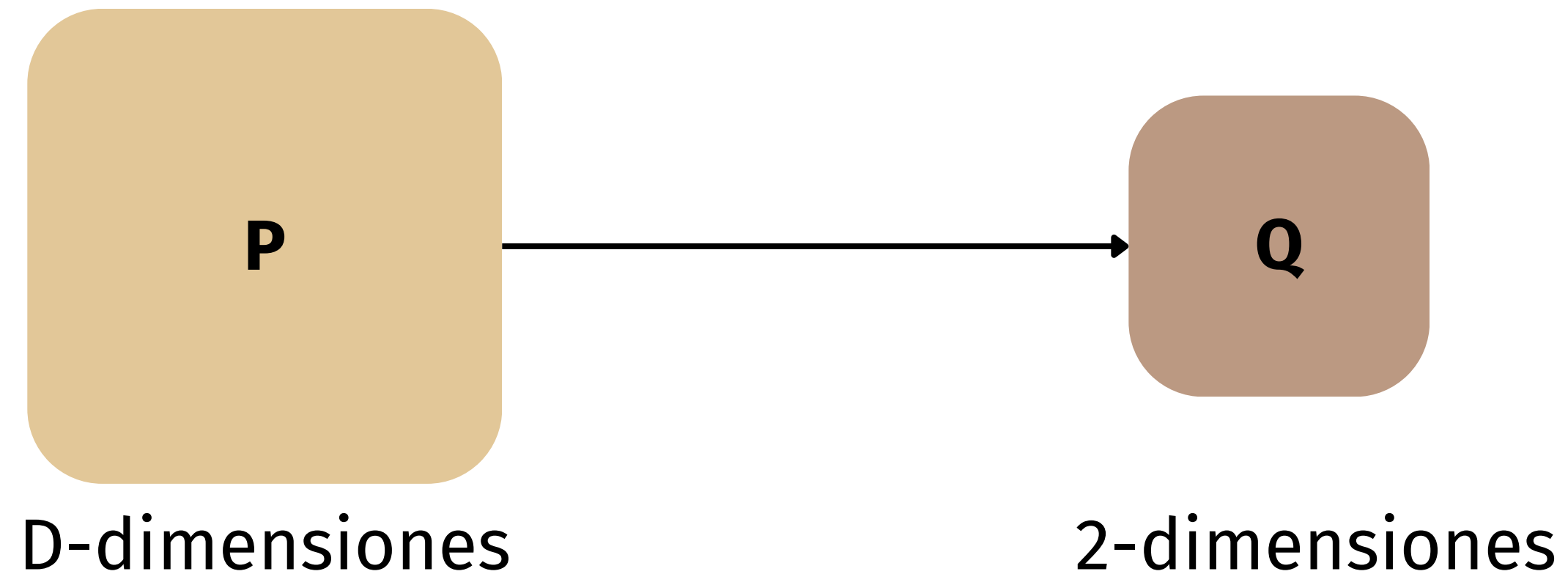
Dados puntos de alta dimensión $\mathbf{x}_i \in \mathbb{R}^D (i = 1, \dots, n)$, t-SNE busca encontrar representaciones que conserven vecindades locales $\mathbf{y}_i \in \mathbb{R}^d$

Esto nos lleva al siguiente problema. Como hacer que dos distribuciones de similitud $\mathbf{P}$ (en alta dimensión) y $\mathbf{Q}$ (en baja dimensión) sean lo más parecidas posible



Stochastic Neighbor Embedding (SNE)

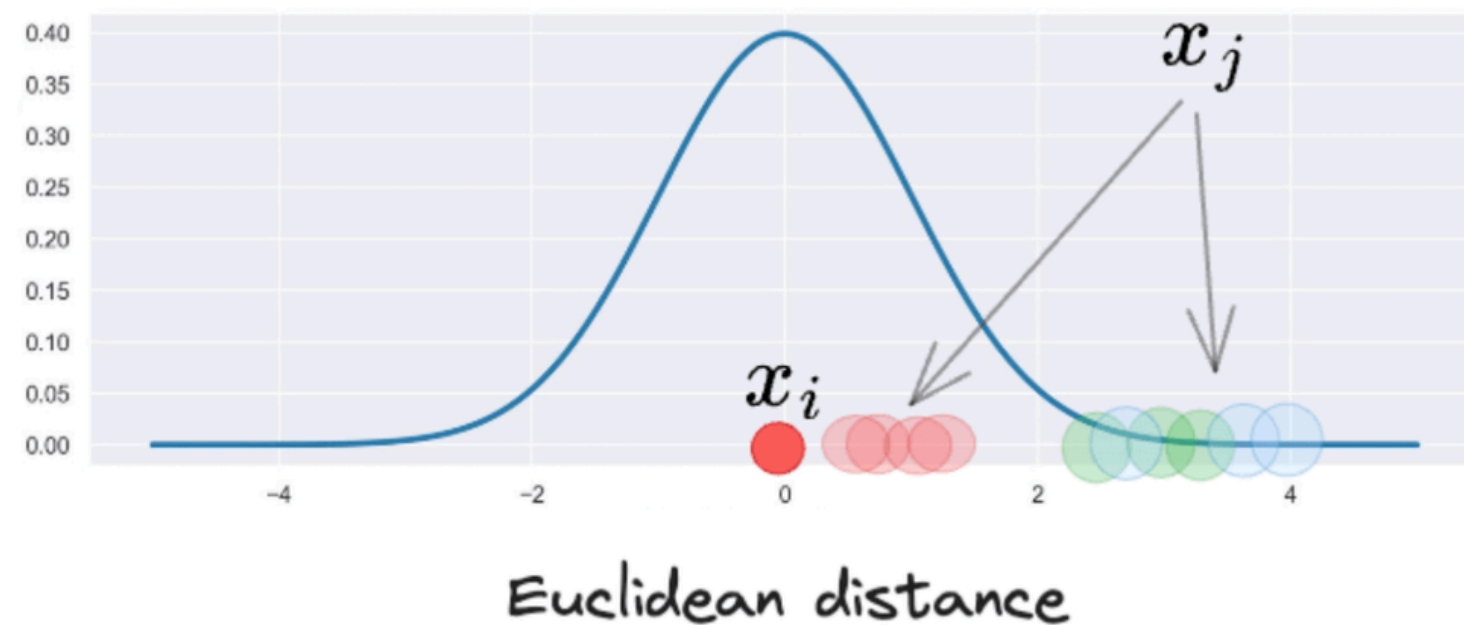
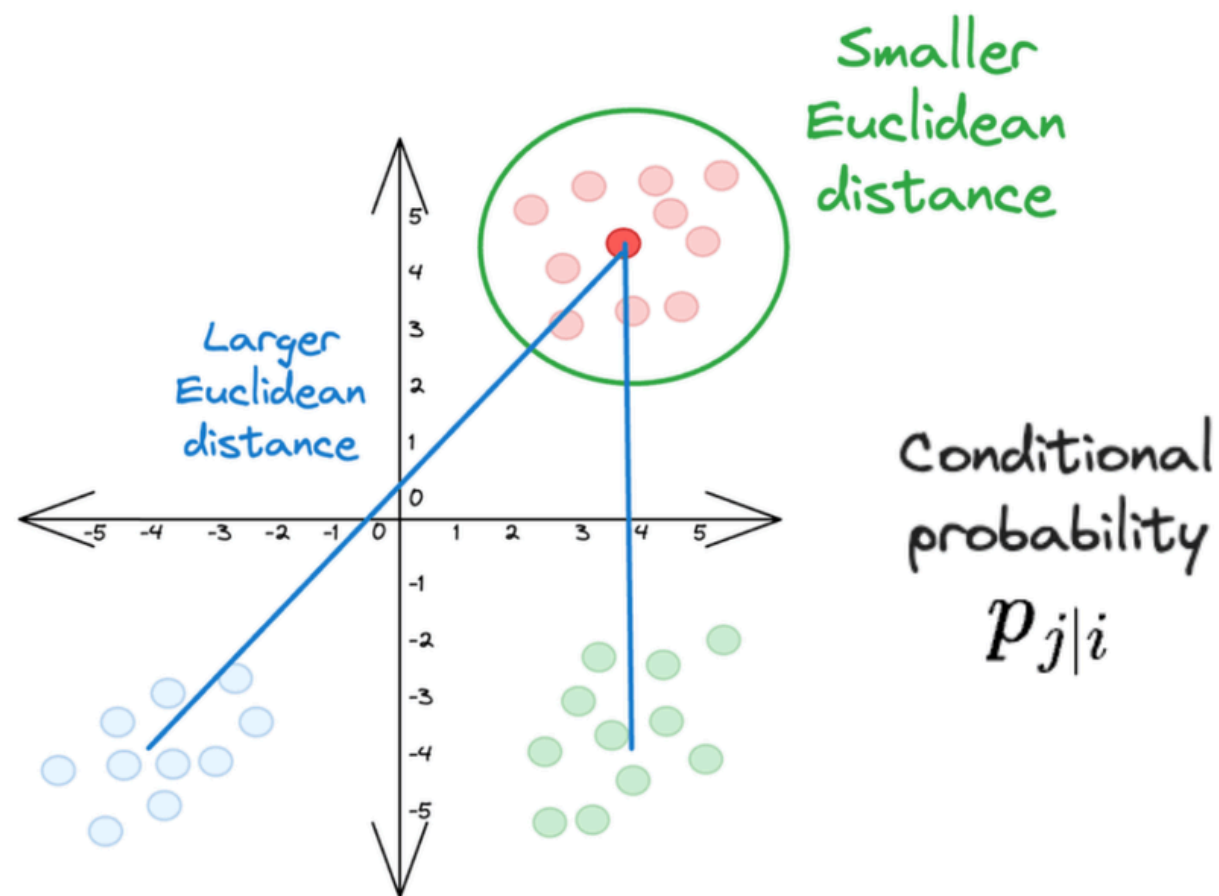
Visualización que **preserva distancias** del espacio original



SNE: Similitud en alta dimensión (P)

Para cada punto, definimos una distribución condicional sobre sus vecinos:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad \sigma_i \text{ es el ancho de la gaussiana del punto } i$$



SNE: Similitud en baja dimensión (Q)

Para cada punto, definimos una distribución condicional sobre sus vecinos:

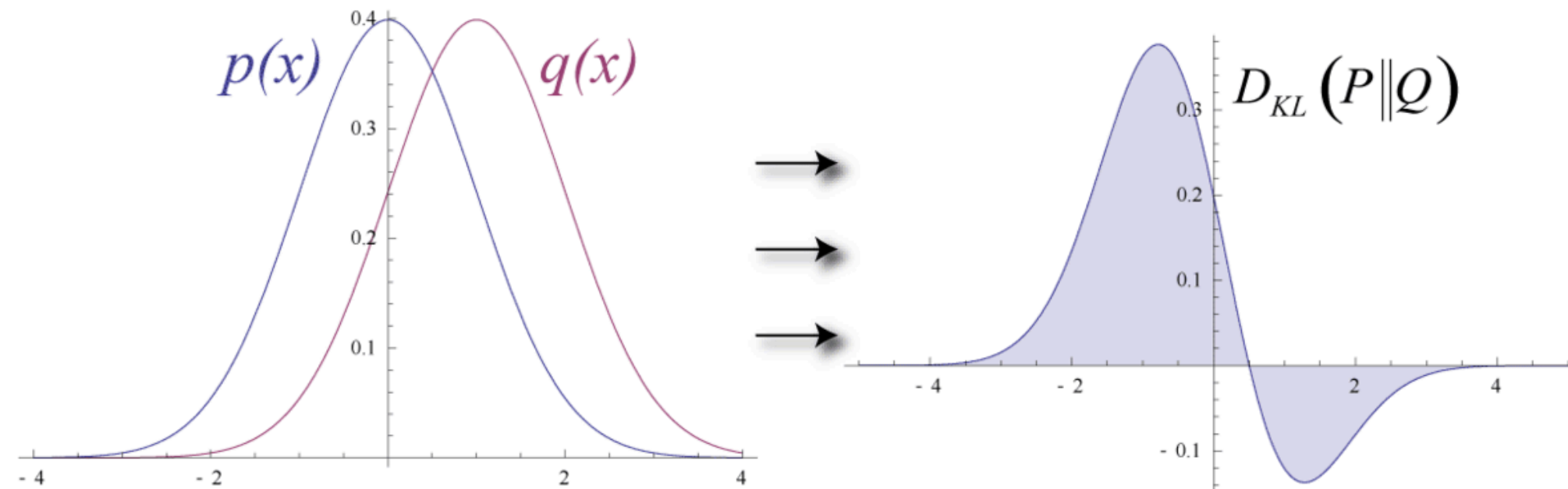
$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)},$$

En baja dimensión no hay sigma, el embedding debe ser coherente en escala para todos los puntos → se fija un σ global

Divergencia KL (Kullback-Leibler)

Determina en qué medida una distribución de probabilidad se desvía de otra distribución de referencia

$$\text{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$



Divergencia KL en SNE

Determina en qué medida una distribución de probabilidad se desvía de otra distribución de referencia

$$\mathcal{C} = \sum_i \text{KL}(P_i \parallel Q_i) = \sum_i \sum_j p_{j|i} \log \left(\frac{p_{j|i}}{q_{j|i}} \right)$$

Debemos minimizar este “costo” de forma que la diferencia entre ambas distribuciones sea la menor posible.

t-SNE

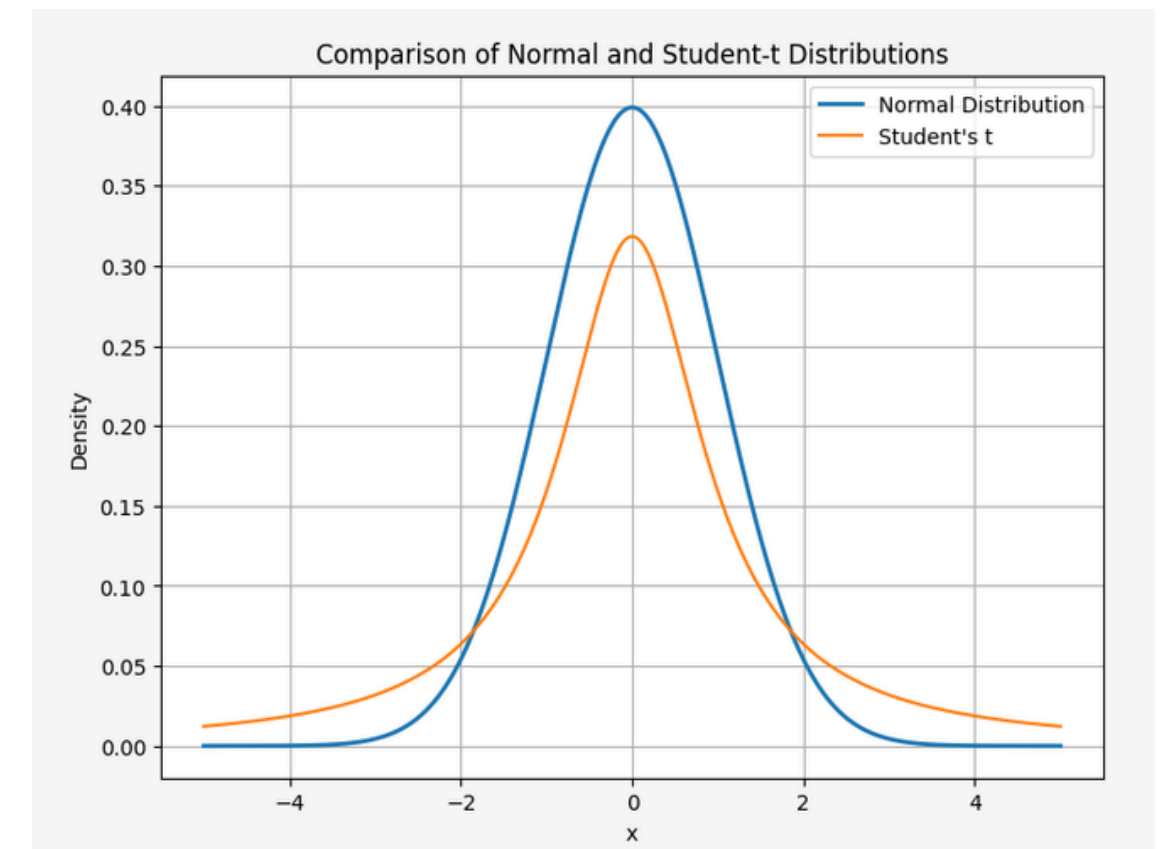
t-SNE (2008, van der Maaten & Hinton) nace como una mejora directa de SNE. Dos diferencias claves:

- En baja dimensión cambia a una **t Student** con 1 grado de libertad

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq \ell} (1 + \|y_k - y_\ell\|^2)^{-1}}$$

- Define distribuciones simétricas

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$



Model Complexity

Entropía: Mide la **incertidumbre promedio** o la cantidad de información esperada en una distribución de probabilidad.

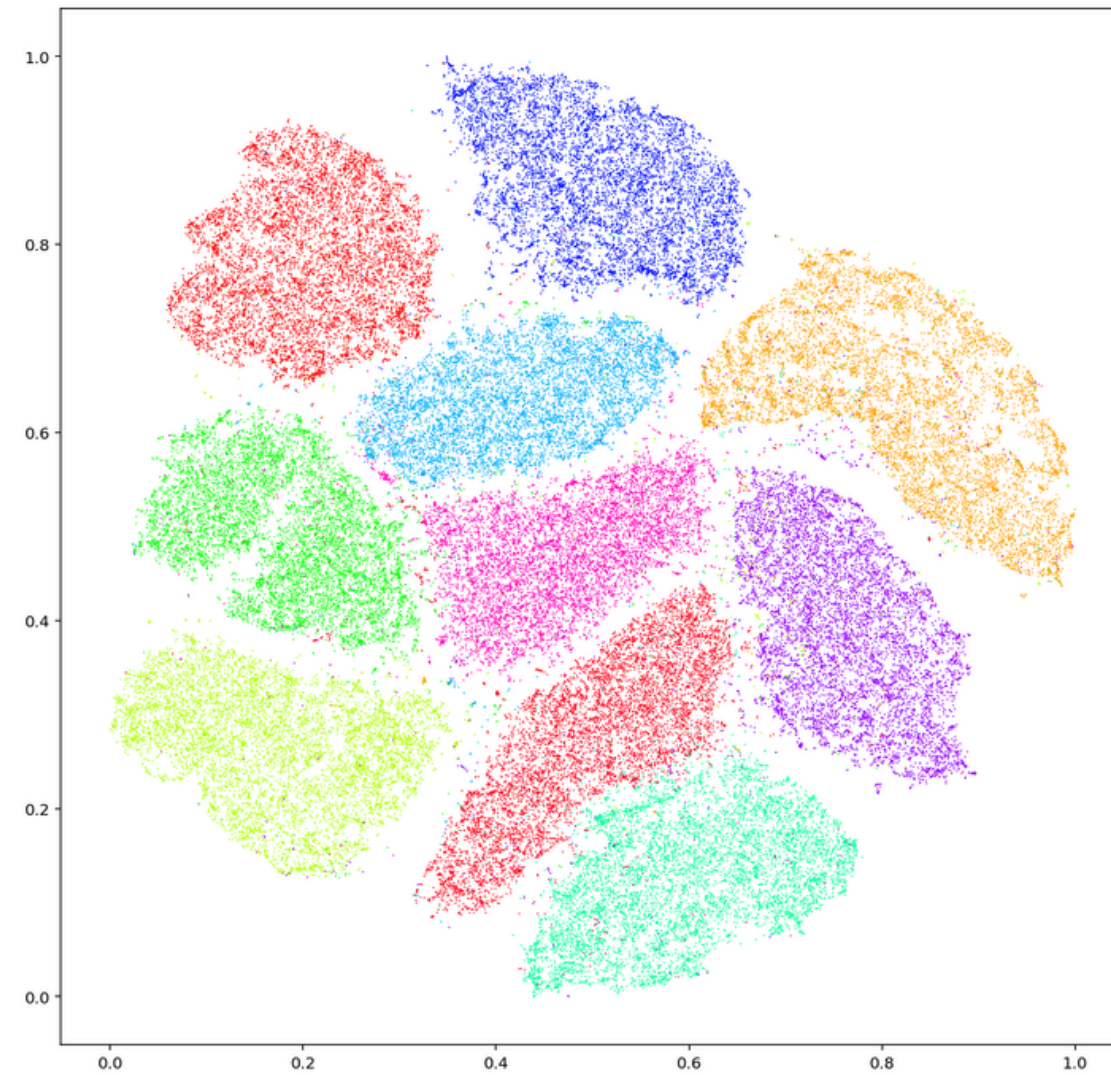
$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

$$\text{Perp}(P_i) = 2^{H(P_i)}$$

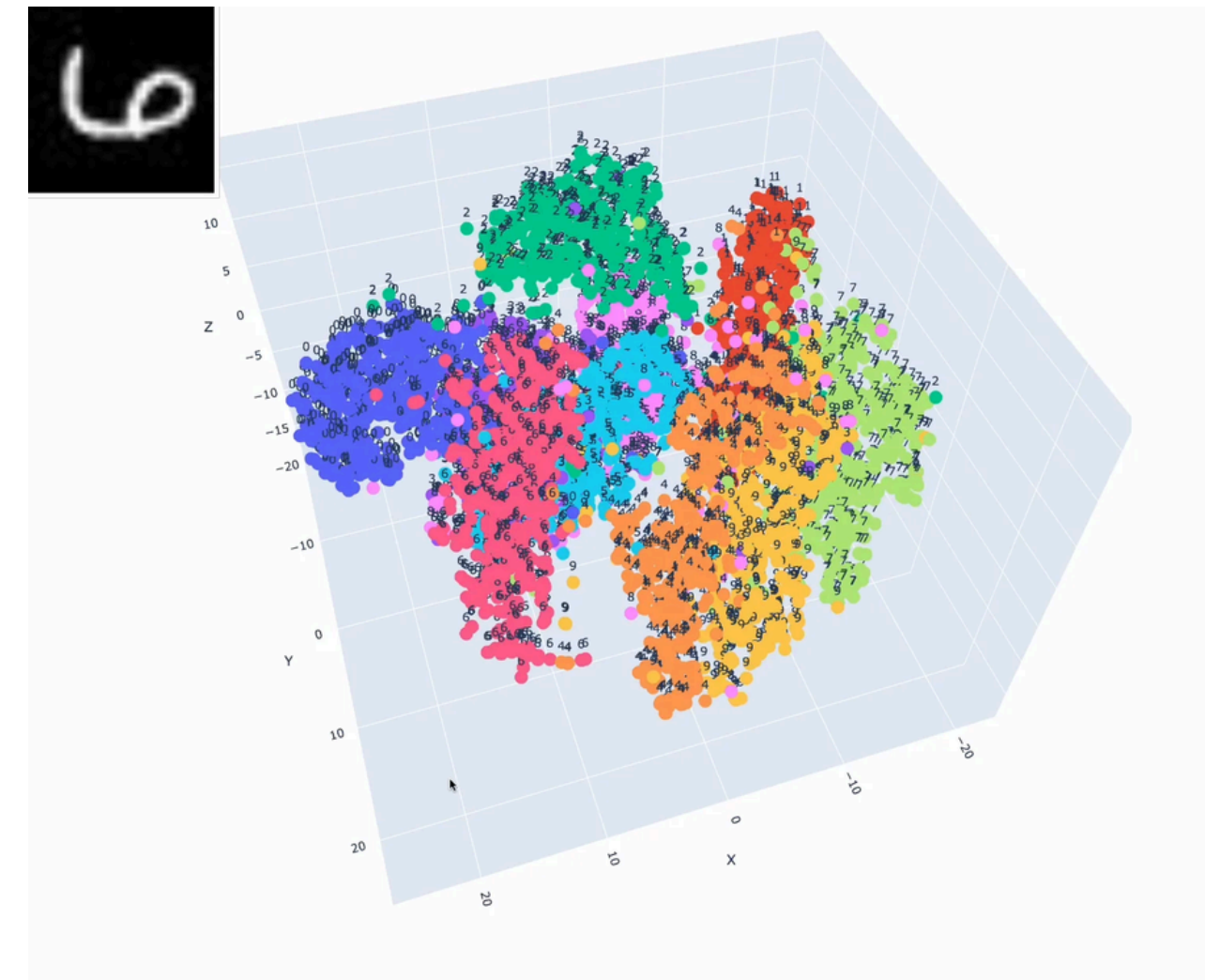
- Se fija una perplejidad objetivo **Perp > 0** (típicamente 5–50)
- Se halla cada σ_i tal que $\text{Perp}(P_i) \approx$ la perplejidad objetivo.

```
tsne = TSNE(n_components=2, perplexity=30)  
X_embedded = tsne.fit_transform(X_scaled)
```

t-SNE



t-SNE 2 dimensiones MNIST

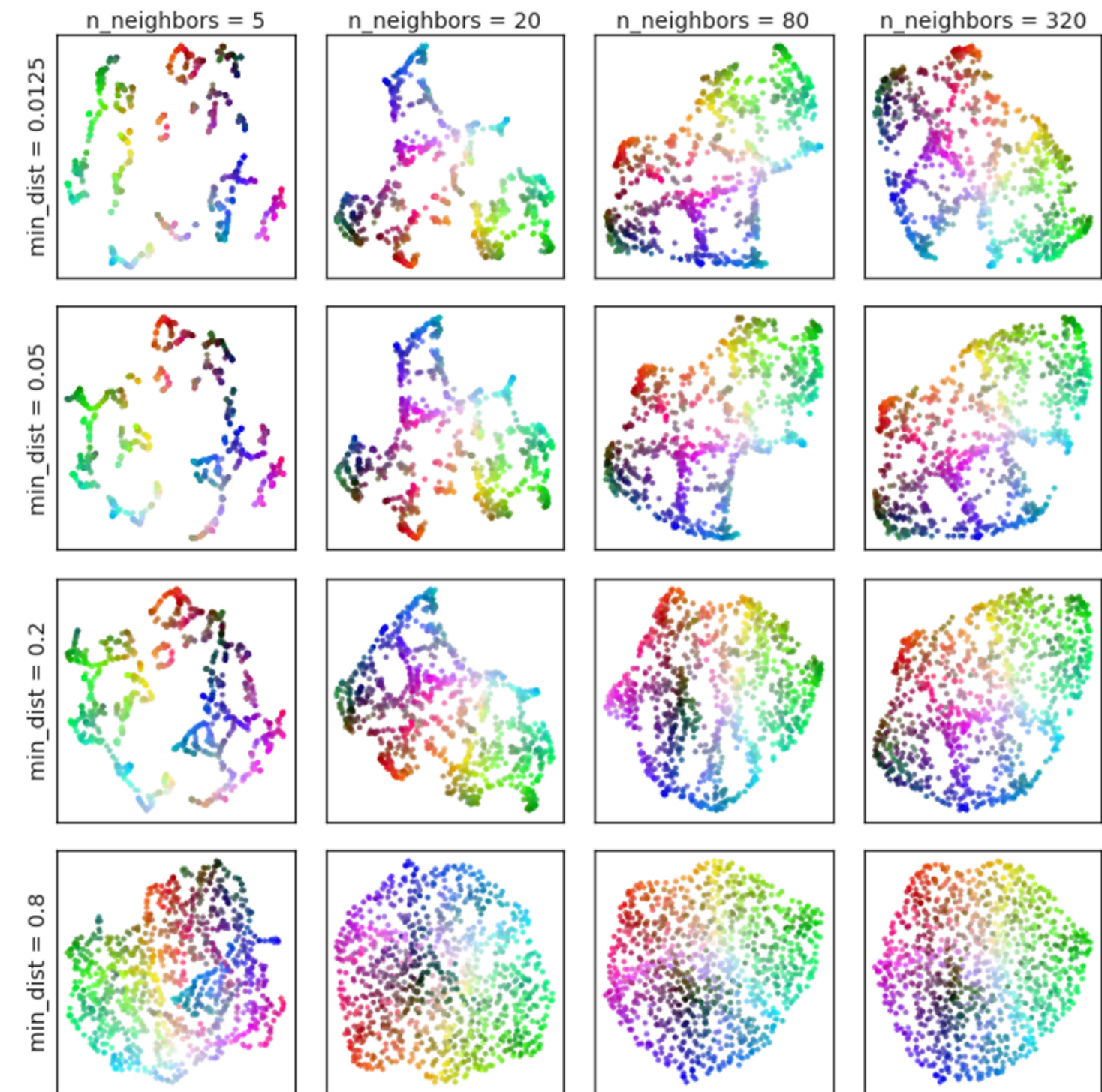


t-SNE 3 dimensiones MNIST

Uniform Manifold Approximation and Projection (UMAP)

Es un algoritmo de reducción de dimensionalidad parecido a t-SNE, pero con una teoría más sólida por debajo:

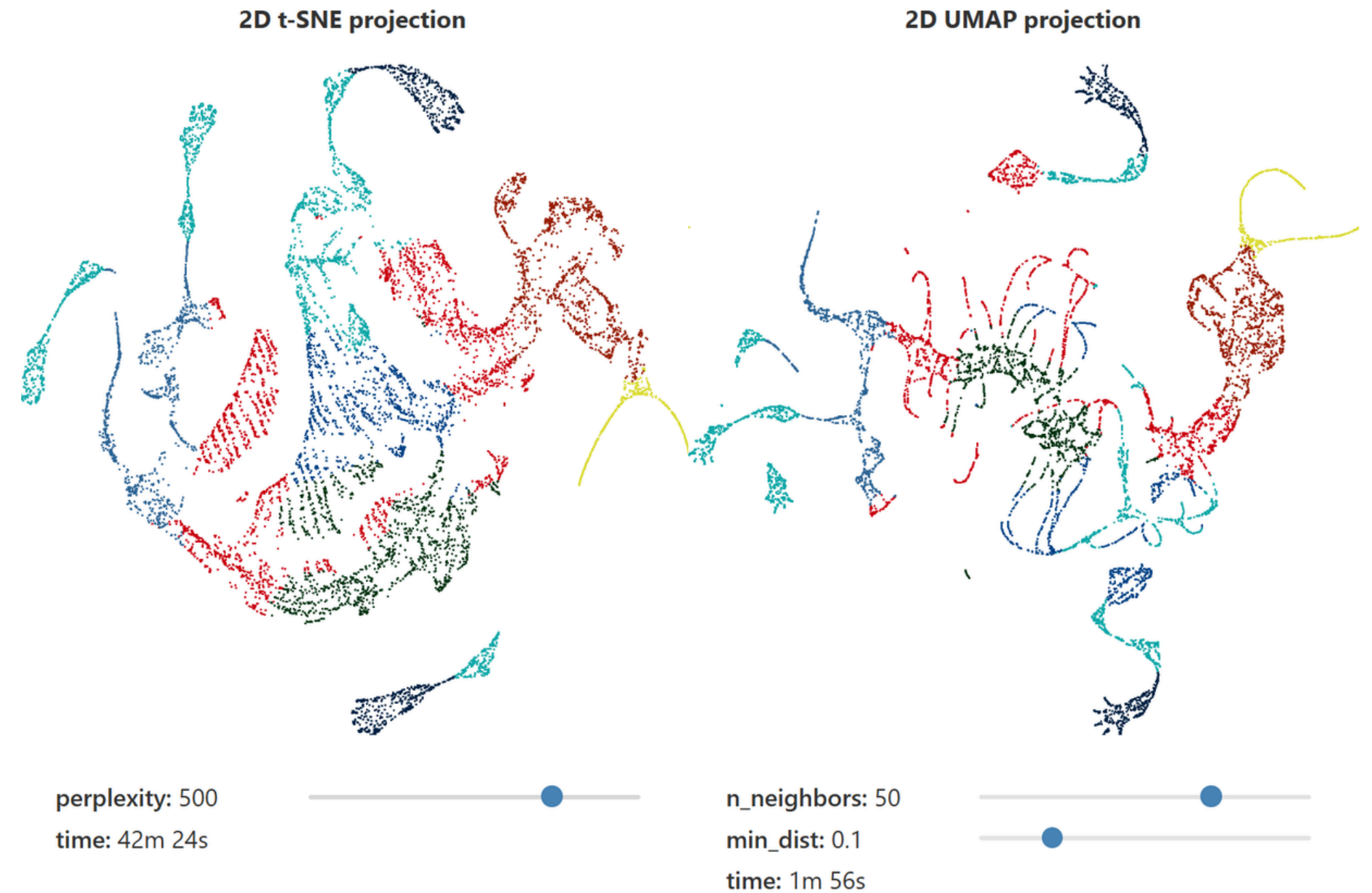
- **Supuesto de variedad:** los datos de alta dimensión X viven en una variedad (manifold) de dimensión mucho más baja dentro de $\mathbb{R}^D$.
- **Aproximación fuzzy-topológica:** la estructura de vecinos se representa con un grafo difuso (fuzzy graph) que codifica las relaciones de cercanía entre puntos.



Uniform Manifold Approximation and Projection (UMAP)

Diferencias con t-SNE:

- **Modelo teórico:** Se basa en geometría de variedades y topología algebraica
- **Estructura global:** Preserva mejor la forma global de los datos.
- **Parámetros clave:**
 - **n_neighbors:** vecinos considerados para aproximar métrica local
 - **min_dist:** separación entre puntos cercanos.





Métricas de Clasificación

K-NN: Clasificación

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1) Dataset
X, y = make_moons(n_samples=1500, noise=0.35, random_state=42)

# 2) Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, stratify=y)

# 3) Estandarización
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# 5) Entrenar k=3
knn = KNeighborsClassifier(n_neighbors=3, metric="euclidean")
knn.fit(X_train_std, y_train)

# 6) Evaluación
y_pred = knn.predict(X_test_std)
y_proba = knn.predict_proba(X_test_std)[:,1]

print("Reporte de clasificación:")
print(classification_report(y_test, y_pred, digits=2))

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred))
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.86	0.87	0.86	150
1	0.86	0.85	0.86	150
accuracy			0.86	300
macro avg	0.86	0.86	0.86	300
weighted avg	0.86	0.86	0.86	300

Matriz de confusión:

```
[[130  20]
 [ 22 128]]
```

Métricas Clasificación

Accuracy: mide la proporción de predicciones correctas sobre el total.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Qué tanto acerté en general

Precision: indica qué proporción de los predichos como positivos realmente lo son.

$$\text{Precision} = \frac{TP}{TP + FP}$$

de lo que predije como X, ¿cuánto realmente era X?

Recall: indica qué proporción de los positivos reales fueron correctamente detectados.

$$\text{Recall} = \frac{TP}{TP + FN}$$

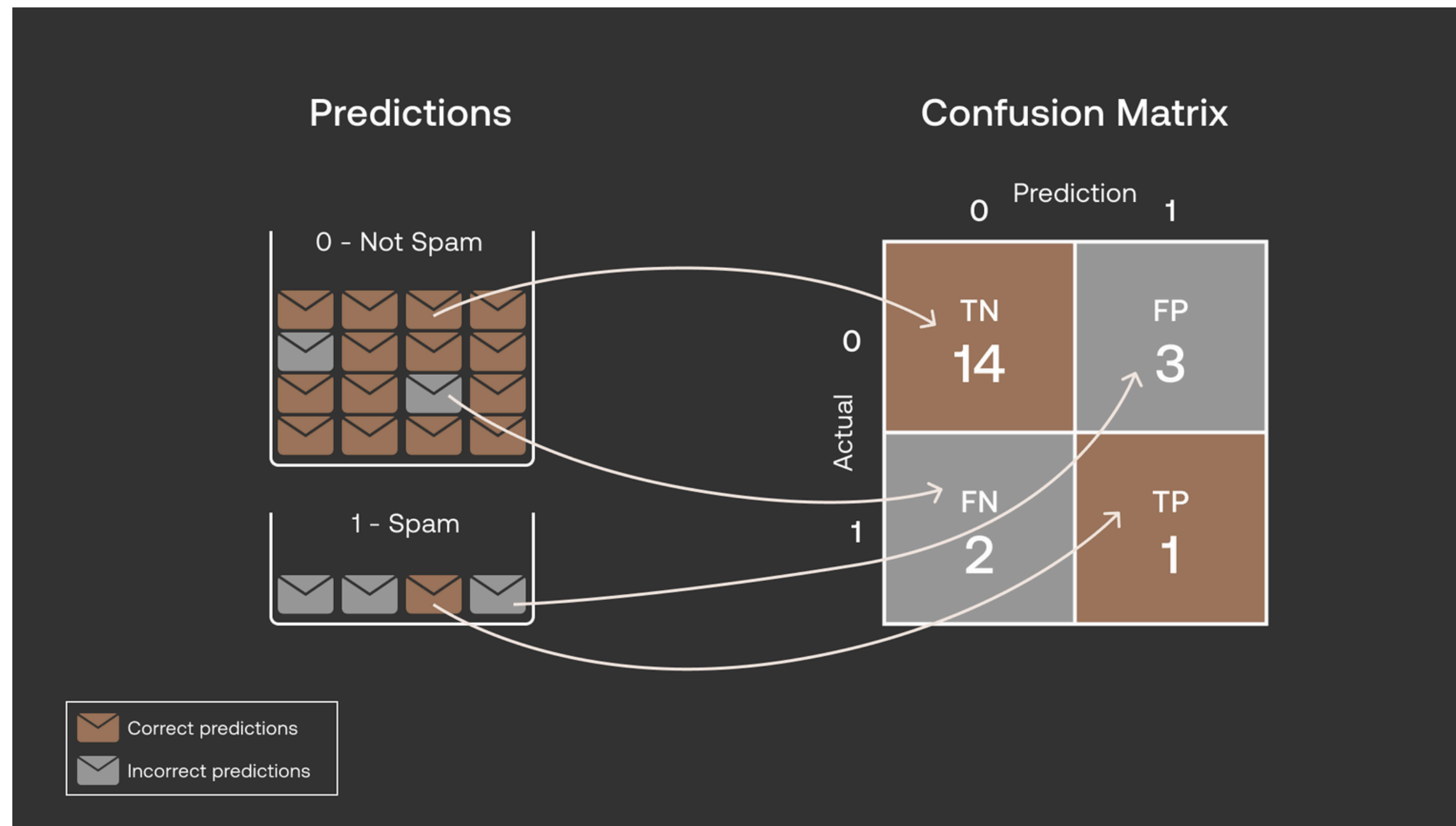
de lo que realmente era X, ¿cuánto detecté?

F1-score: es la media armónica entre precisión y recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

equilibrio entre ambos.

Métricas Clasificación



Las siguientes imágenes fueron sacadas de este link: [Classification Metrics Explained: Accuracy, Precision, Recall & F1 Score](#)

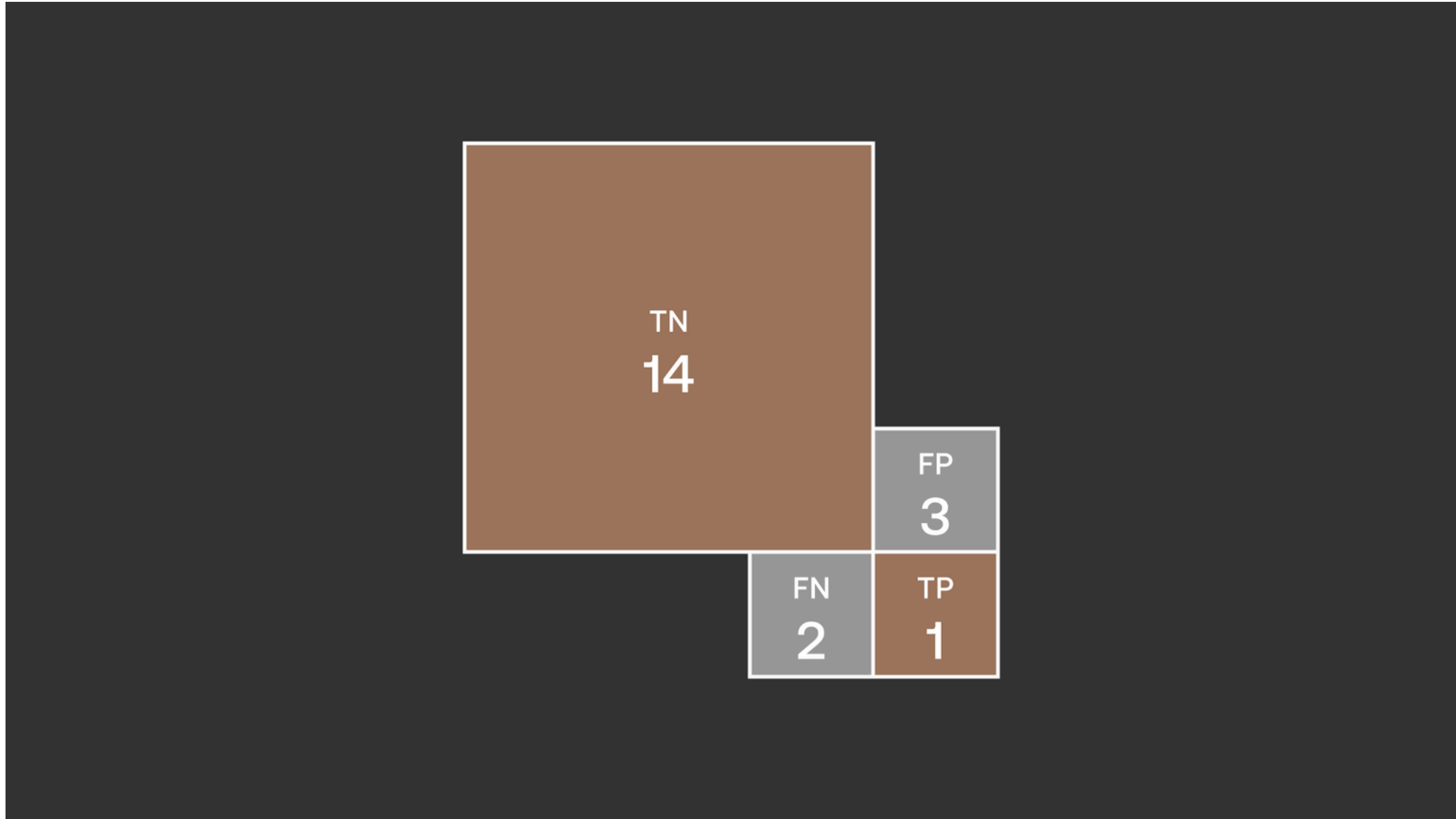
Métricas Clasificación

Accuracy

$$\frac{\text{Num. of correct predictions}}{\text{Total num. of predictions}}$$

$$\frac{\begin{array}{|c|} \hline \text{TN} \\ \hline 14 \\ \hline \end{array} + \begin{array}{|c|} \hline \text{TP} \\ \hline 1 \\ \hline \end{array}}{\begin{array}{|c|} \hline \text{TN} \\ \hline 14 \\ \hline \end{array} + \begin{array}{|c|} \hline \text{TP} \\ \hline 1 \\ \hline \end{array} + \begin{array}{|c|} \hline \text{FN} \\ \hline 2 \\ \hline \end{array} + \begin{array}{|c|} \hline \text{FP} \\ \hline 3 \\ \hline \end{array}} = 75\%$$

Métricas Clasificación



Métricas Clasificación

Precision

$$\frac{\text{True Positives (TP)}}{\text{True Positives (TP) + False Positives (FP)}}$$

		Prediction	
		0	1
Actual	0	TN 14	FP 3
	1	FN 2	TP 1



$$\frac{\begin{array}{|c|} \hline \text{TP} \\ \hline 1 \\ \hline \end{array}}{\begin{array}{|c|} \hline \text{TP} \\ \hline 1 \\ \hline \end{array} + \begin{array}{|c|} \hline \text{FP} \\ \hline 3 \\ \hline \end{array}} = 25\%$$

Métricas Clasificación

Recall

$$\frac{\text{True Positives (TP)}}{\text{True Positives (TP) + False Negatives (FN)}}$$

		Prediction	
		0	1
Actual	0	TN 14	FP 3
	1	FN 2	TP 1



$$\frac{\text{TP } 1}{\text{TP } 1 + \text{FN } 2} = 33\%$$

Métricas Clasificación

Scenario A
High Precision, Low Recall

		Prediction	
		0	1
Actual	0	TN 14	FP 1
	1	FN 4	TP 1

More spam emails went undetected →

Precision: 50%
Recall: 20%

Scenario B
High Recall, Low Precision

		Prediction	
		0	1
Actual	0	TN 14	FP 4
	1	FN 1	TP 1

← More non-spam emails flagged as spam

Precision: 20%
Recall: 50%

Métricas Clasificación

F1

$$\frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 * 25\% * 33\%}{25\% + 33\%} = 28\%$$